WORKFLOWS
LINTING...
TESTING...
DEPLOYING...
GITHUB
RELEASE
DEPENDABOT
SECURITY UPDATES
CI/CD
TRIGGERS
JOBS
STEPS
Real Python

— FREE Email Series —

🐍 Python Tricks 💌

```
# How to merge two dicts
# in Python 3.5+

>>> x = {'a': 1, 'b': 2}
>>> y = {'b': 3, 'c': 4}

>>> z = {**x, **y}

>>> z
{'c': 4, 'a': 1, 'b': 3}
```

Email…

Get Python Tricks »

🔒 No spam. Unsubscribe any time.

Browse Topics · Guided Learning Paths
Basics · Intermediate · Advanced

ai algorithms api best-practices career community databases data-science data-structures data-viz devops django docker editors flask front-end gamedev gui machine-learning news numpy projects python stdlib testing tools web-dev web-scraping

Continuous Integration and Deployment for Python With GitHub Actions

by Ricky White · Nov 27, 2024 · 38m · 0 Comments
advanced devops

Mark as Completed · Share

Table of Contents

- Unlocking the Benefits of CI/CD
- Exploring GitHub Workflows
  - Anatomy of a Workflow File
  - Workflow Triggers
  - Workflow Jobs
  - Workflow Steps
- Using GitHub Actions for Python
  - Exploring the GitHub Marketplace
  - Including Actions in Workflows
- Creating Your First Workflow
- Creating an Automated Testing Workflow
  - Testing on Multiple Versions of Python
  - Finalizing the Testing Workflow
- Publishing Your Package Automatically to PyPI
  - Setting Up and Building the Package
  - Keeping Your Secrets Secure
  - Deploying Your Package
  - Accessing GitHub Environment Variables
- Automating Security and Dependancy Updates
- Next Steps
- Conclusion

Remove ads

Recommended Course
Python Continuous Integration and Deployment Using GitHub Actions (39m)

Creating software is an achievement worth celebrating. But software is never static. Bugs need to be fixed, features need to be added, and security demands regular updates. In today's landscape, with agile methodologies dominating, robust DevOps systems are crucial for managing an evolving codebase. That's where GitHub Actions shine, empowering Python developers to automate workflows and ensure their projects adapt seamlessly to change.

GitHub Actions for Python empowers developers to automate workflows efficiently. This enables teams to maintain software quality while adapting to constant change.

Continuous Integration and **Continuous Deployment** **(CI/CD)** systems help produce well-tested, high-quality software and streamline deployment. GitHub Actions makes CI/CD accessible to all, allowing automation and customization of workflows directly in your repository. This free service enables developers to execute their software development processes efficiently, improving productivity and code reliability.

In this tutorial, you'll learn how to:

- Use **GitHub Actions** and **workflows**
- Automate **linting, testing, and deployment** of a Python project
- **Secure credentials** used for automation
- Automate **security and dependency updates**

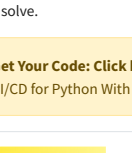
This tutorial will use an existing codebase, Real Python Reader, as a starting point for which you'll create a CI/CD pipeline. You can fork the Real Python Reader code on GitHub to follow along. Be sure to deselect the *Copy the* `master` *branch only* option when forking. Alternatively, if you prefer, you can build your own Real Python Reader using a previous tutorial.

In order to get the most out of this tutorial, you should be comfortable with `pip`, building Python packages, Git, and have some familiarity with YAML syntax.

Before you dig into GitHub Actions, it may be helpful to take a step back and learn about the benefits of CI/CD. This will help you understand the kinds of problems that GitHub Actions can solve.

Get Your Code: Click here to download the free sample code you'll use to learn about CI/CD for Python With GitHub Actions.

Take the Quiz: Test your knowledge with our interactive "GitHub Actions for Python" quiz. You'll receive a score upon completion to help you track your learning progress:



Interactive Quiz
GitHub Actions for Python
In this quiz, you'll test your understanding of GitHub Actions for Python. By working through this quiz, you'll revisit how to use GitHub Actions and workflows to automate linting, testing, and deployment of a Python project.

Unlocking the Benefits of CI/CD

Continuous Integration (CI) and **Continuous Deployment (CD)**, commonly known as **CI/CD**, are essential practices in modern software development. These practices automate the integration of code changes, the execution of tests, and the deployment of applications. This helps teams and open-source contributors to deliver code changes more frequently in a reliable and structured way.

Moreover, when publishing open-source Python packages, CI/CD will ensure that all pull requests (PRs) and contributions to your package will meet the needs of the project while standardizing the code quality.

Note: To learn more about what a pull request is and how to create one, you can read GitHub's official documentation.

More frequent deployments with smaller code changes **reduce the risk of unintended breaking changes** that can occur with larger, more complex releases. For example, even though developers can format all code using the same linting tools and rules, policy can automatically block PRs from being merged if the code's tests don't pass.

In the next section, you'll learn how GitHub Workflows can help you implement CI/CD on a repository hosted on GitHub.

Remove ads

Exploring GitHub Workflows

GitHub Workflows are a powerful feature of GitHub Actions. They allow you to define custom automation workflows for your repositories. Whether you want to build, test, or deploy your code, GitHub Workflows provide a flexible and customizable solution that any project on GitHub can use for free, whether the repository is public or private.

Even though there are many CI/CD providers, GitHub Actions has become the default among open-source projects on GitHub because of its expansive ecosystem, flexibility, and low or no cost.

Anatomy of a Workflow File

Workflow files are declaratively written YAML files with a predefined structure that must be adhered to for a workflow to run successfully. Your YAML workflow files are stored and defined in a `.github/workflows/` folder in your project's root directory.

Your workflow folder can have multiple workflow files, each of which will perform a certain task. You can name these workflow files anything you'd like. However, for the sake of simplicity and readability, it's common practice to name them after the tasks they achieve, such as `test.yml`.

Each file has a few elements that are required, but many, many more that are optional. The GitHub Actions documentation is thorough and well-written, so be sure to check it out after you've finished reading this tutorial.

There are three main parts that make up the bulk of a workflow file: **triggers**, **jobs**, and **steps**. You'll cover these in the next sections.

Workflow Triggers

A trigger is an event that causes a workflow to run. There are many kinds of triggers. The most common ones are those that occur on a:

- **Pull request**
- **Pushed commit** to the default branch
- **Tagged commit**
- **Manual** trigger
- Request by **another workflow**
- **New issue** being opened

You might also want to restrict triggers further by limiting it to a specific branch or set of files. Here's a simple example of a trigger that runs a workflow on *any* push to the main branch:

YAML `.github/workflows/example.yml`

```yaml
on:
  push:
    branches:
      - main
```

For detailed information about the triggers not covered in this tutorial, you can check out the official documentation.

Now that you know how events trigger workflows, it's time to explore the next component of a workflow file: jobs.

Workflow Jobs

Each workflow has a single `jobs` section, which is the container for the meat and potatoes of the workflow. A workflow can include one or more jobs that it will run, and each job can contain one or more steps.

Here's an example of what this section would look like *without* any steps:

YAML `.github/workflows/example.yml`

```yaml
# ...

jobs:
  my_first_job:
    name: My first job
  my_second_job:
    name: My second job
```

When you're creating a job, the first thing to do is define the runner you want to use to run your job. A `runner` is a GitHub-hosted virtual machine (VM) that executes your jobs for you. GitHub will provision and de-provision the VM so you don't have to worry about maintaining any infrastructure for your CI/CD.

There are multiple supported operating systems available. You can find the full list of GitHub-hosted runners in the documentation.

Note: Self-hosted runners are also an option if the free and unlimited versions don't meet your needs. This tutorial doesn't cover self-hosted runners, but you can find detailed information about using self-hosted runners in the official documentation.

Defining a runner takes as little as a single line of YAML:

YAML `.github/workflows/example.yml`

```yaml
# ...

jobs:
  my_first_job:
    name: My first job
    runs-on: ubuntu-latest
    # ...
  my_second_job:
    name: My second job
    runs-on: windows-latest
    # ...
```

In the above example, `my_first_job` will run inside an Ubuntu VM, and `my_second_job` will run inside a Windows VM. Both use the `-latest` suffix in this case, but you could also specify the exact version of the operating system—for example, `ubuntu-20.24`, as long as it's a supported version.

Remove ads

Workflow Steps

Steps are the main part of a job. As you've probably guessed, the steps declare the actions that need to be performed when executing the workflow. This can include tasks such as installing Python, running tests, linting your code, or using another GitHub action.

Just like your Python code, common and repeatable tasks can be abstracted away into separate workflows and reused. This means you can and should use other people's GitHub Actions in your own workflows, similar to how you would when importing a Python library, to save you time reimplementing that functionality.

In the next section, you'll see how you can use other GitHub Actions and how to find them.

Using GitHub Actions for Python

Even though workflows are a part of GitHub Actions, workflows can also contain GitHub Actions. In other words, you can use other people's or organization's actions in your workflow. In fact, it's common practice and highly encouraged to use existing GitHub Actions in your workflow files. This practice saves you time and effort by leveraging pre-built functionalities.

If you have a specific task to accomplish, there's likely a GitHub Action available to do it. You can find relevant GitHub Actions in the GitHub Marketplace, which you'll dive into next.

Exploring the GitHub Marketplace

The GitHub Marketplace is an online repository of all the actions people can use in their own workflows. GitHub, third-party vendors, and individuals build and maintain these GitHub Actions. Anyone can use the GitHub Action template to create their own action and host it in the marketplace.

This has led to a vast array of GitHub Actions available for nearly every type of task automation imaginable. All actions in the GitHub Marketplace are open source and free to use.

In the next section, you'll look at two GitHub Actions that you'll use for every Python project.

Including Actions in Workflows

Every Python-based workflow you create needs to not only check out your current repository into the workflow environment but also install and set up Python. Fortunately, GitHub has official GitHub Actions to help with both tasks:

YAML `.github/workflows/example.yml`

```yaml
# ...

jobs:
  my_first_job:
    name: My first job
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.13"
      - run: python -m pip install -r requirements.txt
```

In the example above, you can see that the first step in `steps` is to use the official `checkout` action. This action checks out the code from your repository into the current GitHub workspace, allowing your workflow to access it. The `@4` following `checkout` is a version specifier, indicating which version of the action to use. As of now, the latest version is v4.2.2, so you can refer to it using this syntax to specify the latest major version.

The second step of this example sets up Python in the environment. Again, this example uses the official GitHub Action to do this because of its ongoing support and development. Most actions, if not all, have extra configurations you can add to the step.

The Setup Python action documentation contains the complete list of configurations. For now, the minimum you need to install Python into your workflow environment is to declare which version of Python you wish to install.

In the final step of the example, you use the `run` command. This command allows you to execute any `bash` or `powershell` command, depending on which runner you're using for the step. In this case, you're installing the project's dependencies from the requirements file.

Hopefully, you can see how powerful GitHub Actions can be. With very little code and effort, you have a reproducible way to set up an environment that's ready for building, testing, and deploying your Python project.

You now have a basic understanding of the structure of a workflow file and how you can create your first workflow for a project. In the next section, you'll do just that with a real-world example.

Remove ads

Creating Your First Workflow

It's time to walk through the steps of adding CI/CD to an existing real-world project, the Real Python Reader. Before you add workflows for testing and deploying this package, you should first start with linting.

A linter is a tool that analyzes your code and looks for errors, stylistic issues, and suspicious constructs. Linting allows you to address issues and improve your code quality before you share it with others. By starting your CI/CD with linting, you'll ensure that your code is clean and readable before deploying the package to PyPI.

Note: If linting is a new concept for you, then you can learn more about it by reading about Ruff, a modern Python linter.

For this workflow, you'll use Ruff to lint the Python code. But if you haven't already, first fork the repository, including all branches, and then clone it. Be sure to replace *your-username* with your GitHub username:

Shell

```shell
$ git clone git@github.com:your-username/reader.git
$ cd reader/
$ git checkout github-actions-tutorial
$ mkdir -p .github/workflows/
```

After you clone your forked repository and change your current working directory, you'll need to switch to the pre-existing branch named `github-actions-tutorial`. If such a branch is unavailable, then you most likely forgot to uncheck the *Copy the* `master` *branch only* option when forking. In such a case, you should delete your fork, go back to the original repository, fork it again, and ensure that you include all branches this time.

Once you've successfully switched to the correct branch, create a folder to store your workflows. This folder should be called `workflows/` and be a subdirectory of the `.github/` folder.

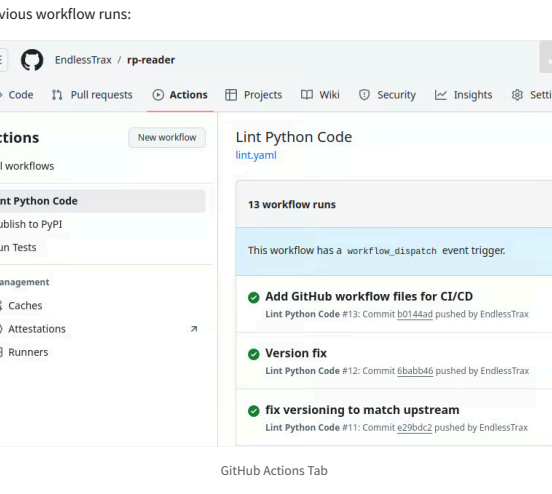
Note: When you fork a repository that has existing GitHub Actions, you might see a prompt asking you to enable them after you click on the *Actions* tab of your forked repository. This is a safety feature. By confirming that you want to enable the actions, you won't have any issues following the rest of this tutorial.

Now, you're ready to create your first workflow where you'll define your triggers, set up the environment, and install Ruff. To start, you can define your triggers in the `lint.yml` file:

YAML `.github/workflows/lint.yml`

```yaml
name: Lint Python Code

on:
  pull_request:
    branches:
      - master
  push:
    branches:
      - master
  workflow_dispatch:
```

Even though it's not required, it's considered best practice to give each of your workflows a clear, human-readable name. This name will appear in the left column of the *Actions* tab on your GitHub repository. It helps you identify the available workflows and filter through your previous workflow runs:



13 workflow runs

This workflow has a `workflow_dispatch` event trigger.

Add GitHub workflow files for CI/CD
Lint Python Code #13: Commit b014da0 pushed by EndlessTrax

Version fix
Lint Python Code #12: Commit 6ba6bb6 pushed by EndlessTrax

fix versioning to match upstream
Lint Python Code #11: Commit e29bdc2 pushed by EndlessTrax

GitHub Actions Tab

After defining the name, you can shift your focus to the triggers for this workflow. In the code above, there are three different triggers defined that can initiate the workflow:

1. Opening a pull request
2. Pushing local commits
3. Dispatching the workflow manually

The first two will trigger the workflow on any push or pull request event on the `master` branch. This means that any change to the code will trigger this workflow to run, whether you push straight to `master`, or use a pull request to merge code into the `master` branch on your repository.

Note: This workflow gets triggered by events on the `master` branch while you're working on another branch. If you'd like to see the action take effect immediately after pushing your commits to GitHub, then consider adding `github-actions-tutorial` to the list of branches monitored by the workflow.

It's not obvious what the final trigger does. According to the documentation, it's commonly used to rerun a workflow that failed for reasons unrelated to code changes, such as an expired API key. However, the `workflow_dispatch` trigger only works when the workflow file is on the default branch.

With the triggers defined, it's time to proceed to the next step in creating the workflow file, which is to define the jobs and configure the environment:

YAML `.github/workflows/lint.yml`

```yaml
name: Lint Python Code

on:
  pull_request:
    branches:
      - master
  push:
    branches:
      - master
  workflow_dispatch:

jobs:
  lint: # The name of the job
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.13"
          cache: "pip"
```

Most of this code should look familiar from earlier examples, but there are a couple of small differences. First, you've named the job `lint` to describe what it does. This is just a name, so you can choose any name you like as long as it adheres to YAML syntax. You've also defined the runner you'll be using for this workflow as `ubuntu-latest`.

Then, you'll notice that the `setup-python` action is now configured to cache the pip dependencies of any installed packages. This helps speed up your workflow in future runs if the versions of a package are the same. Instead of pulling them from PyPI, it will use the cached versions.

Note: To learn more about how you can use caching in your workflows, you can check out the GitHub documentation.

Now that your workflow has a defined trigger and runner, and with your code checkout and Python installed, it's time to install Ruff and run it to lint the code. You can do this by adding two more steps to your `lint` job:

YAML `.github/workflows/lint.yml`

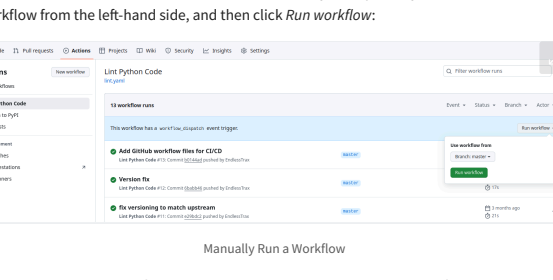
```yaml
name: Lint Python Code

on:
  pull_request:
    branches:
      - master
  push:
    branches:
      - master
  workflow_dispatch:

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.13"
          cache: "pip"
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          python -m pip install ruff

      - name: Run Ruff
        run: ruff check --output-format=github
```

In the last two steps of the `lint` job, you use the `run` command that you saw earlier. As part of the YAML syntax, you'll notice a pipe (`|`) symbol on the second line. This denotes a multi-line string. The `run` command will interpret the following lines as separate commands and execute them in sequence.

After installing Ruff, the workflow finally finishes by running Ruff to look for linting errors. With this command, you can specify that you want the output to be optimized for running in a `github` workflow with the `--output-format` tag.

Note: If you use Ruff and have your own configurations outside of the default, you might replace these last two steps with Ruff's own GitHub Action.

Congratulations! You've completed your first workflow. Once this workflow is committed to your repository and pushed, GitHub will automatically run this linting workflow when the trigger condition is met. You may also trigger this workflow manually at anytime on the GitHub website. To do this, head to the *Actions* tab on your repository, select the desired workflow from the left-hand side, and then click *Run workflow*:



Manually Run a Workflow

Now that you have a workflow under your belt and understand how workflows work, it's time to create one that runs the test suite on Real Python Reader.

Remove ads

Creating an Automated Testing Workflow

Now that you've gotten your feet wet with your first GitHub workflow, it's time to look at what will arguably be the most important of all of the workflows for this package: automated testing.

The Real Python Reader uses `pytest` as its testing framework. And given what you've already learned about GitHub Actions, you might even see how you can edit the linting workflow to turn it into a testing workflow. After all, you're going to follow the same steps to get ready to run `pytest`. It's important to note that when you're testing a software package, you should test it on all supported versions of Python.

But first, as with all GitHub workflows, you need to declare the triggers for the testing workflow:

YAML `.github/workflows/test.yml`

```yaml
name: Run Tests

on:
  push:
    branches:
      - master
  pull_request:
    branches:
      - master
  workflow_call:
  workflow_dispatch:
```

Much of the above is the same as the previous linting workflow but with one difference—there's now a new trigger, `workflow_call`. Much like `workflow_dispatch`, `workflow_call` is a predefined trigger that lets other workflows trigger this workflow.

This means that if you have a workflow in the future that also requires the tests to pass, instead of repeating the code, you can ask the new workflow to use this testing workflow. The workflow will then trigger this testing workflow as one of its steps, and ensure it passes before moving on to the job's other steps. So no more repetition, and you can keep your workflows shorter and to the point.

Although you won't be using this method of workflow reuse in your `test.yml` workflow, you would achieve this in the same way you call other GitHub Actions in your workflow file, by using the `uses` keyword:

YAML

```yaml
# github-username/repo/path/to/workflow@version
- uses: realpython/reader/.github/workflows/test.yml@master
```

Here, you can see that you can reuse a workflow by passing a path-like string to `uses`. It should start with the GitHub username and repository name, followed by the path to the workflow file you want to use. `@master` tells the new workflow that you want to use the version of the testing workflow from the `master` branch. And now, you can see how powerful GitHub Actions can be. Reusing workflows is a huge advantage of GitHub Actions.

Now that you've defined the triggers for the testing workflow, it's time to address the question: How do you test on multiple versions of Python? In the next section, you'll see how you can define your steps once and have them run multiple times, with each run being on a different version of Python.

Testing on Multiple Versions of Python

In the linting workflow, you used the `setup-python` action in your `steps` to set up Python 3.13 in the Ubuntu instance, which looked like this:

YAML `.github/workflows/lint.yml`

```yaml
# ...

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.13"
          cache: "pip"
        # ...
# ...
```

Unfortunately, you can't just add a list of Python versions to `python-version` and be done. What you need is a `strategy` matrix to test on multiple versions of Python.

To quote the official documentation:

> A matrix strategy lets you use variables in a single job definition to automatically create multiple job runs that are based on the combinations of the variables. For example, you can use a matrix strategy to test your code in multiple versions of a language or on multiple operating systems. (Source)

In short, whatever variables you define in your `matrix` will run the same steps in the job, but using those variables. Here, you want to run on different versions of Python, but you could also use this to run or build your code on different operating systems.

Declaring a strategy is relatively straightforward. Before defining your steps but as part of your job, you can define your required strategy:

YAML `.github/workflows/test.yml`

```yaml
name: Run Tests

on:
  push:
    branches:
      - master
  pull_request:
    branches:
      - master
  workflow_call:
  workflow_dispatch:

jobs:
  testing:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ["3.9", "3.10", "3.11", "3.12", "3.13"]
```

As you can see, you're declaring a variable `python-version`, which is an array of version numbers. Great, this is the first part done! The second part is to tell the `setup-python` action that you want to use these versions using a special variable syntax:

YAML `.github/workflows/test.yml`

```yaml
name: Run Tests

on:
  push:
    branches:
      - master
  pull_request:
    branches:
      - master
  workflow_call:
  workflow_dispatch:

jobs:
  testing:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ["3.9", "3.10", "3.11", "3.12", "3.13"]

    steps:
      - uses: actions/checkout@v4
      - name: Set up Python ${{ matrix.python-version }}
        uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}
          cache: "pip"
```

The Python setup step of the workflow now has two changes. The first is the added `name` to the step. As you learned earlier, this isn't required but it will help you identify which Python version failed by referencing the Python version in the step's name. This is helpful, given that this step will run for five different versions of Python.

The second change is that instead of hard coding the version number into the `with: python-version` part of `setup-python`, you can now refer to the `python-version` defined in the matrix.

GitHub has a few special contexts that you can access as part of your workflows. Matrix is one of these. By defining the matrix as part of the strategy, `python-version` has now become a property of the matrix context. This means that you can access any variable defined as part of the matrix with the dot (`.`) syntax, for example, `matrix.python-version`.

Although this isn't something that needs to be done for Real Python Reader, you could do the same with different OS versions. For example:

YAML

```yaml
strategy:
  matrix:
    os: [ubuntu-latest, windows-latest]
```

You could then use the same dot notation to access the `os` variable you defined in the matrix with `matrix.os`.

Now that you know how to use a matrix to run your steps declaratively using a different version of Python, it's time to complete the testing workflow in full.

Remove ads

Finalizing the Testing Workflow

There are just a few more steps needed in order to finish the workflow. Now that Python is installed, the workflow will need to install the developer dependencies and then finally run `pytest`.

The Real Python Reader package uses a `pyproject.toml` configuration file for declaring its dependencies. It also has optional developer dependencies, which include `pytest`. You can install them the same way you installed Ruff earlier, using the `run` command:

YAML `.github/workflows/test.yml`

```yaml
name: Run Tests

on:
  push:
    branches:
      - master
  pull_request:
    branches:
      - master
  workflow_call:
  workflow_dispatch:

jobs:
  testing:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ["3.9", "3.10", "3.11", "3.12", "3.13"]

    steps:
      - uses: actions/checkout@v4
      - name: Set up Python ${{ matrix.python-version }}
        uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}
          cache: "pip"

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          python -m pip install .[dev]
```

This step is all you need to install the required dependencies. The only remaining step is to run `pytest`:

YAML `.github/workflows/test.yml`

```yaml
name: Run Tests

on:
  push:
    branches:
      - master
  pull_request:
    branches:
      - master
  workflow_call:
  workflow_dispatch:

jobs:
  testing:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ["3.9", "3.10", "3.11", "3.12", "3.13"]

    steps:
      - uses: actions/checkout@v4
      - name: Set up Python ${{ matrix.python-version }}
        uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}
          cache: "pip"

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          python -m pip install .[dev]

      - name: Run Pytest
        run: pytest
```

At this point, you have both a linting and testing workflow that are triggered whenever a PR or push event happens on master. Next, you'll turn your attention to the CD part of CI/CD, and learn how you can automatically publish a package to PyPI.

Publishing Your Package Automatically to PyPI

The third workflow rounds off what most people view as a minimum CI/CD pipeline. This third workflow provides a reproducible and consistent way to build and publish a package. The Real Python Reader package utilizes the widely-used Python `build` library to generate Python distribution files, which can then be deployed to PyPI.

When workflows get a little more complicated and have multiple steps or jobs, it's recommended that you write out the steps and flow. This will help you get all the steps in the right order so that the GitHub Actions you use are configured correctly from the start. This will save you time later by helping you avoid potential bugs in your build workflow.

Here are the workflow steps for the `deploy.yml` file:

1. **Set up the environment** by installing Python and build dependencies
2. **Build the package** by placing output files in a `dist/` folder
3. **Publish the distribution files** to PyPI
4. **Create a GitHub release** if published successfully

In the next section, you'll tackle the first two items on the list and have a good portion of your workflow written.

Setting Up and Building the Package

As with the past two workflows, the first step is to define the triggers for the workflow. You've seen some common triggers that revolve around typical developer workflows, but automatically releasing with every new PR or push to the main branch isn't ideal for Real Python Reader.

It makes more sense to bump the version of the package after several pull requests, bug fixes, or after adding new features. The modern way of triggering such a release after a version bump is to use the developer's best friend, Git.

Git allows you to tag a commit to denote a notable point in time in the software's development. This is often the tool of choice to define a new release. GitHub Actions have built-in support for using Git tags as triggers through the `tags` keyword:

YAML `.github/workflows/deploy.yml`

```yaml
name: Publish to PyPI
on:
  push:
    tags:
      - "*.*.*"
```

As you can see here, triggers also support glob patterns. So an asterisk (`*`) can match any character in a sequence. The pattern outlined above will match any character followed by a decimal point (`.`), another character, another decimal point, and finally, another character.

This means that 1.0.0 is a valid match, as is 2.5.60. This matches the semantic versioning used by Real Python Reader. You could also use `v*.*.*` instead if you prefer. So, your Git tags should start with a `v`, which stands for *version*. For example, `v1.0.0` would be a valid tag.

In order to trigger this workflow, you'd tag a commit with the version name:

Shell

```shell
$ git tag -a "1.0.0" -m "1.0.0"
$ git push --tags
```

Pushing your new tag to GitHub will then trigger this workflow. Next, you'll set up the environment and install the dependencies:

YAML `.github/workflows/deploy.yml`

```yaml
name: Publish to PyPI
on:
  push:
    tags:
      - "*.*.*"

jobs:
  publish:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.13"

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          python -m pip install .[build]

      - name: Build package
        run: python -m build
```

First, you define the `publish` job and install Python 3.13 into an Ubuntu VM. The next step installs the build dependencies of Real Python Reader. In the last step, you use the same `run` command you've used before, but this time, instead of running Ruff or `pytest`, you'll build the Real Python Reader package. By default, `build` will place the distribution files in a folder called `dist`.

Excellent! You've implemented the first two main parts of the workflow plan. Before you can deploy to PyPI, you should know how to keep your PyPI API token secure.

Remove ads

Keeping Your Secrets Secure

As you learned earlier, workflows get access to special contexts like `matrix`. Another context that all workflows have access to is the `secrets` context. By storing sensitive data as a **repository secret**, you can ensure you never accidentally leak API keys, passwords, or other credentials. Your workflow can access those sensitive credentials using the `secrets` context.

You can add secrets to your repository on the GitHub website. Once you've added them, you can't view or edit them. You can only replace them with a new value. It's a good idea to review the GitHub documentation to see how to add secrets on the GitHub website. The official docs are continually updated with any UI changes, making them the best source for learning how to use this GitHub feature.

Deploying Your Package

After securing your API key as a GitHub secret, you can access it in the workflow:

YAML `.github/workflows/deploy.yml`

```yaml
name: Publish to PyPI
on:
  push:
    tags:
      - "*.*.*"

jobs:
  publish:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.13"

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          python -m pip install .[build]

      - name: Build package
        run: python -m build

      - name: Test publish package
        uses: pypa/gh-action-pypi-publish@release/v1
        with:
          user: __token__
          password: ${{ secrets.PYPI_API_TOKEN }}
          repository-url: https://test.pypi.org/legacy/

      - name: Publish package
        uses: pypa/gh-action-pypi-publish@release/v1
        with:
          user: __token__
          password: ${{ secrets.PYPI_API_TOKEN }}
```

In this step, you get to use the official GitHub Action from the Python Packaging Authority (PyPA), which manages PyPI. This GitHub Action does most of the work and only needs a reference to your PyPI API token. Again, by default, it will look in your `dist` folder for any new version of a package to upload.

Rather than using a traditional username and password to authenticate to PyPI, it's best practice to use a scoped API token instead for automatic releases.

Since you're using an API token and there's no username, using `__token__` as the username tells the GitHub Action that token authentication is being used. Just like with the previous matrix strategy, you can use dot notation to access the secret context, as in `secrets.PYPI_API_TOKEN`.

The name of the secret when stored in GitHub doesn't matter, as long as it makes sense to you. The GitHub secret is named `PYPI_API_TOKEN`, so you reference it inside the workflow using that name.

You may have noticed that the workflow includes a test step prior to publishing the package to PyPI. This step is almost identical to the publishing step, with one key difference: you'll need to provide a `repository-url` to override the default URL and push the package to test.pypi.org.

Using TestPyPI is an excellent way to ensure that your package is built and versioned correctly. It allows you to identify and address any potential issues that might cause problems when publishing to the main PyPI repository.

If you're following along with your own fork of the repository and intend to push your version to PyPI, then you'll need to update the name of the project to a unique name. If you don't update the project name, you'll recieve an HTTP 403 error when trying to upload it. This is because you don't have permission to publish the `realpython-reader` package to PyPI. Updating the project name will allow you to publish your own version.

As an example, you could add your username as a prefix to the project name:

TOML `pyproject.toml`

```toml
[build-system]
requires      = ["setuptools>=61.0.0", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "username-realpython-reader"
# ...
```

There's just one more step of the workflow to complete—creating a GitHub release to promote and then sharing the release directly. Before you can do this, you'll learn about GitHub environment variables.

Accessing GitHub Environment Variables

In order to publish a release to a GitHub repo, a GitHub token is required. You may have used these before if you've ever used the GitHub API. Given the security risk of using personal GitHub tokens in workflows, GitHub creates a read-only token in the secrets context by default. This means that you always have access to it if you need it.

In addition, every GitHub runner includes the handy GitHub CLI by default. This makes performing certain tasks, like creating a release, so much simpler. The GitHub CLI has many ways to authenticate the user, one of which is by setting an environment variable called `GITHUB_TOKEN`.

You may see where this is going. The provided GitHub token can be used to access the CLI and ultimately create a seamless way to create the GitHub release. Here's what that would look like in the workflow:

YAML `.github/workflows/deploy.yml`

```yaml
 1  name: Publish to PyPI
 2  on:
 3    push:
 4      tags:
 5        - "*.*.*"
 6
 7  jobs:
 8    publish:
 9      runs-on: ubuntu-latest
10      steps:
11        - uses: actions/checkout@v4
12        - name: Set up Python
13          uses: actions/setup-python@v5
14          with:
15            python-version: "3.13"
16
17        - name: Install dependencies
18          run: |
19            python -m pip install --upgrade pip
20            python -m pip install .[build]
21
22        - name: Build package
23          run: python -m build
24
25        - name: Test publish package
26          uses: pypa/gh-action-pypi-publish@release/v1
27          with:
28            user: __token__
29            password: ${{ secrets.PYPI_API_TOKEN }}
30            repository-url: https://test.pypi.org/legacy/
31
32        - name: Publish package
33          uses: pypa/gh-action-pypi-publish@release/v1
34          with:
35            user: __token__
36            password: ${{ secrets.PYPI_API_TOKEN }}
37
38        - name: Create GitHub Release
39          env:
40            GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
41          run: |
42            gh release create ${{ github.ref_name }} ./dist/* --generate-notes
```

You'll see that on lines 39 and 40, the workflow specifically assigns the GitHub token from the secrets context to an environment variable called `GITHUB_TOKEN`. Any key values set in `env` will be set as environment variables for the current step. This means that when you run the GitHub CLI (`gh`), it will have access to the token through the assigned environment variable. The GitHub CLI can't directly access the secrets context itself.

GitHub also lets you access a special context called `github`. The workflow references the `ref_name` attribute in the `github` context. This is defined in the GitHub docs as follows:

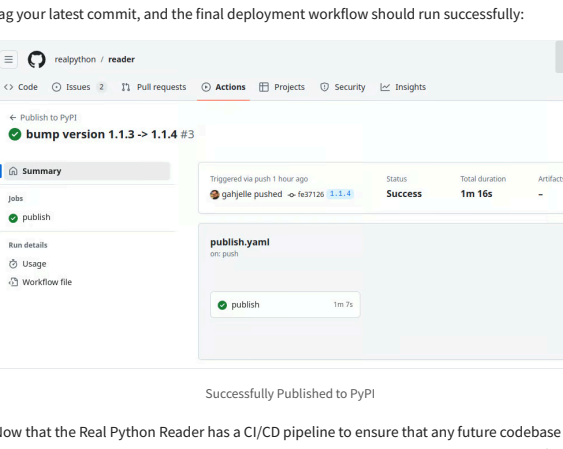
> The short ref name of the branch or tag that triggered the workflow run. (Source)

So, `github.ref_name` will be replaced with the attribute that triggered the workflow, which in this case is the Git tag's name.

The `gh` command above will create a release with the same name as the tag used to trigger the release, upload all files from `./dist`, and auto-generate release notes. These release notes include any PRs that developers have merged since they created the last release, giving proper credit to the authors with links and usernames for their contributions.

You may want to add any missing details to the release notes. Remember that releases can be edited after creation if you need to include additional information, such as deprecation notices.

Congratulations! You now have automated linting, testing, and deployment in place. You can tag your latest commit, and the final deployment workflow should run successfully:



bump version 1.1.3 -> 1.1.4 #3

Triggered via push 1 hour ago
Status: Success
Total duration: 1m 16s
Artifacts: –

publish.yaml
on: push

publish 1m 16s

Successfully Published to PyPI

Now that the Real Python Reader has a CI/CD pipeline to ensure that any future codebase changes are robust and use readable and consistent code, you can add one more workflow to Real Python Reader. The cherry on the top of our CI/CD cake, so to speak.

In the next section, you'll learn how to configure Dependabot to automate security and dependency updates.

Remove ads

Automating Security and Dependancy Updates

Just like Python code, your GitHub workflows need to be maintained and kept up-to-date. Furthermore, the libraries that the Real Python Reader code relies on are always changing and updating, so it's hard to keep up and manage dependencies.

It can be particularly difficult to stay informed about any security updates released by your dependencies if you're not actively following the project on GitHub or social media. Luckily, GitHub has a handy tool to help with both problems. Enter Dependabot!

Dependabot is an automation tool that will not only notify you of a security vulnerability in your dependencies but, if configured, will automatically create a PR to update and fix the issue for you. All you have to do is review the automated PR and merge. With Dependabot, keeping your package up-to-date and free from known security vulnerabilities is quick and easy, saving you time you can use to improve your code or add new features.

You can configure Dependabot to meet the needs of your project. Here, the Real Python Reader package has fairly basic requirements. The two goals are:

1. To be notified when there's a dependency update available.
2. To help keep the other workflows up-to-date.

These requirements are defined in a configuration file called `dependabot.yml`. Unlike the other workflows, the `dependabot.yml` file lives in the `.github` folder itself, *not* in `.github/workflows`.

Because this file is only twelve lines long and you're now more familiar with YAML syntax, you can take a look at the final Dependabot configuration:

YAML `.github/dependabot.yml`

```yaml
---
version: 2
updates:
  - package-ecosystem: "pip"
    directory: "/"
    schedule:
      interval: "weekly"

  - package-ecosystem: "github-actions"
    directory: "/"
    schedule:
      interval: "weekly"
```

The `version` property is a mandatory part of the file. This is where you'll define the version of Dependabot to use, and version 2 is the latest. Another mandatory section is `updates`. This is where the bulk of the configuration goes. Each update defines the package ecosystem to check, along with basic information regarding which directory Dependabot should search in, as well as how often.

For the first update, Dependabot will check common files where `pip` dependencies are typically declared, such as `requirements.txt`, `pyproject.toml`, and others. Since the Real Python Reader has a `pyproject.toml` file in the root directory, Dependabot is instructed to look there, as indicated by the forward slash (`"/"`).

How often you want to be notified of dependency updates is up to you. Each project will have its own requirements. However, having it declared in YAML means that if you find the cadence too much, or not enough, it's a quick and simple change to make. For now, you can use `weekly`.

The second item in the `updates` list is for `github-actions`. That's right, Dependabot will also check the GitHub Actions used in any workflow in the repo, such as `setup-python`, for newer versions! This makes keeping up with the latest versions of GitHub Actions automatic, and is one less thing for you to be concerned about.

Note: There are many more configuration settings you can use with Dependabot, including the option to automatically tag GitHub users for review when it creates a PR. For more information about other configuration options, refer to the official GitHub Docs.

With this configuration in place, Dependabot will scan and check your repository once a week to see if there are any updates you can make to dependencies or your workflows. It will create a PR with a fix automatically. These PRs from Dependabot will also run your other workflows to make sure that Dependabot's changes pass your linting and testing checks. Double win!

Next Steps

There are many other tasks you can automate as your repository grows, such as **issue triage, labeling, stale issue management, adding reviewers to PRs**, and more.

Also, keep in mind that GitHub Actions is just one provider of CI/CD. If your project is hosted on GitHub, then GitHub Actions can make things simpler for you. If your code is on another platform or you want to try alternatives, here's a short list of other CI/CD providers:

- GitLab
- Azure Pipelines
- CircleCI
- Travis CI

If you already use one of these providers or one that isn't listed, please feel free to shout it out in the comments and share your experiences.

Remove ads

Conclusion

You now know how to implement a robust CI/CD pipeline for a Python project using GitHub Actions. While the goal of this tutorial was for you to learn how to add CI/CD to an existing codebase, hopefully you now know enough to work with your own projects and packages and create your own workflows from scratch.

In this tutorial, you learned how to:

- Use **GitHub Actions** and **workflows**
- Automate **linting, testing, and deployment** of a Python project
- **Secure credentials** used for automation
- Automate **security and dependency updates**

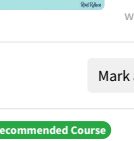
By automating these processes, you've significantly improved the maintainability and reliability of your project. You now have a consistent way to ensure code quality, run tests, and deploy new versions with minimal manual intervention.

Remember that CI/CD is an iterative process. As your project grows and evolves, you may need to adjust your workflows or add new ones. The flexibility of GitHub Actions allows you to adapt easily to changing requirements.

With these tools and practices in place, you're well-equipped to manage and scale your Python projects efficiently.

Get Your Code: Click here to download the free sample code you'll use to learn about CI/CD for Python With GitHub Actions.

Take the Quiz: Test your knowledge with our interactive "GitHub Actions for Python" quiz. You'll receive a score upon completion to help you track your learning progress:



Interactive Quiz
GitHub Actions for Python
In this quiz, you'll test your understanding of GitHub Actions for Python. By working through this quiz, you'll revisit how to use GitHub Actions and workflows to automate linting, testing, and deployment of a Python project.

Mark as Completed · Share

Recommended Course
Python Continuous Integration and Deployment Using GitHub Actions (39m)

🐍 Python Tricks 💌

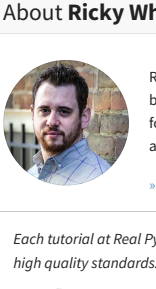
Get a short & sweet **Python Trick** delivered to your inbox every couple of days. No spam ever. Unsubscribe any time. Curated by the Real Python team.

```
# How to merge two dicts
# in Python 3.5+

>>> x = {'a': 1, 'b': 2}
>>> y = {'b': 3, 'c': 4}

>>> z = {**x, **y}

>>> z
{'c': 4, 'a': 1, 'b': 3}
```

Email Address

Send Me Python Tricks »

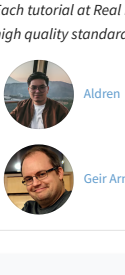
About **Ricky White**



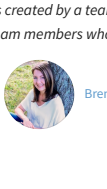
Ricky is a software engineer and writer from a non-traditional background. He's an enthusiastic problem solver with passion for creating and building, from software and websites to books and bonsai.

» More about Ricky

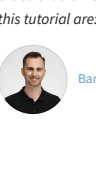
Each tutorial at Real Python is created by a team of developers so that it meets our high quality standards. The team members who worked on this tutorial are:



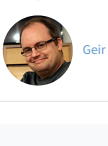
Aldren



Brenda

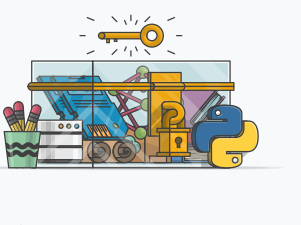


Bartosz



Geir Arne

Master Real-World Python Skills
With Unlimited Access to Real Python



Join us and get access to thousands of tutorials, hands-on video courses, and a community of expert Pythonistas:

Level Up Your Python Skills »

What Do You Think?

Rate this article:

LinkedIn · Twitter · Bluesky · Facebook · Email

What's your #1 takeaway or favorite thing you learned? How are you going to put your newfound skills to use? Leave a comment below and let us know.

Commenting Tips: The most useful comments are those written with the goal of learning from or helping out other students. Get tips for asking good questions and get answers to common questions in our support portal.

Looking for a real-time conversation? Visit the Real Python Community Chat or join the next "Office Hours" Live Q&A Session. Happy Pythoning!

Keep Learning

Related Topics: advanced devops

Related Learning Paths:
- DevOps With Python
- Testing and Continuous Integration

Related Courses:
- Python Continuous Integration and Deployment Using GitHub Actions

Related Tutorials:
- Build Robust Continuous Integration With Docker and Friends
- Build a Hash Table in Python With TDD
- pytest Tutorial: Effective Python Testing

Learn Python: Start Here · Learning Resources · Code Mentor · Python Reference · Python Cheat Sheet · Support Center
Courses & Paths: Learning Paths · Quizzes & Exercises · Browse Topics · Live Courses · Books
Community: Podcast · Newsletter · Community Chat · Office Hours · Learner Stories
Membership: Plans & Pricing · Team Plans · For Business · For Schools · Reviews
Company: About Us · Team · Mission & Values · Editorial Guidelines · Sponsorships · Careers · Press Kit · Merch

Privacy Policy · Terms of Use · Security · Contact
❤ Happy Pythoning!

© 2012–2026 DevCademy Media Inc. DBA Real Python. All rights reserved.
REALPYTHON™ is a trademark of DevCademy Media Inc.